KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

정렬과 sorted, max, min

무엇을 기준으로 줄 세우고
가장 큰 값을 고를지 key로 정하기

```
sales = {"coffee": 12, "latte": 30,
         "cake": 8, "tea": 15}

sorted(sales, key=sales.get, reverse=True)
# ['latte', 'tea', 'coffee', 'cake']

max(sales, key=sales.get)
# 'latte'
```

오늘의 순서

1 왜 필요한가 4–6쪽

2 sorted, sort, key 7–12쪽

3 튜플과 딕셔너리 정렬 13–18쪽

4 max와 min 19–22쪽

5 오류와 함정 23–25쪽

6 실습 26–29쪽

3쪽은 노트북 준비, 30–31쪽은 막힐 때와 한 장 요약이에요.

들어가기

21에서 이어서 해요

✓	리스트, 튜플, sort, sorted, max, min	13
✓	딕셔너리 get, items()	15
✓	sorted(..., reverse=True)	16, 20
✓	for, enumerate()	13, 18
✓	lambda, map, filter	21

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

가장 많이 팔린 메뉴를 물었는데 tea가 나와요

[1]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales))
print(max(sales))
```

```
['cake', 'coffee', 'latte', 'tea']
tea
```

← 기대한 건 latte

← 키만 abc 순으로 줄 섰어요

← abc 순으로 가장 뒤라서 가장 커요

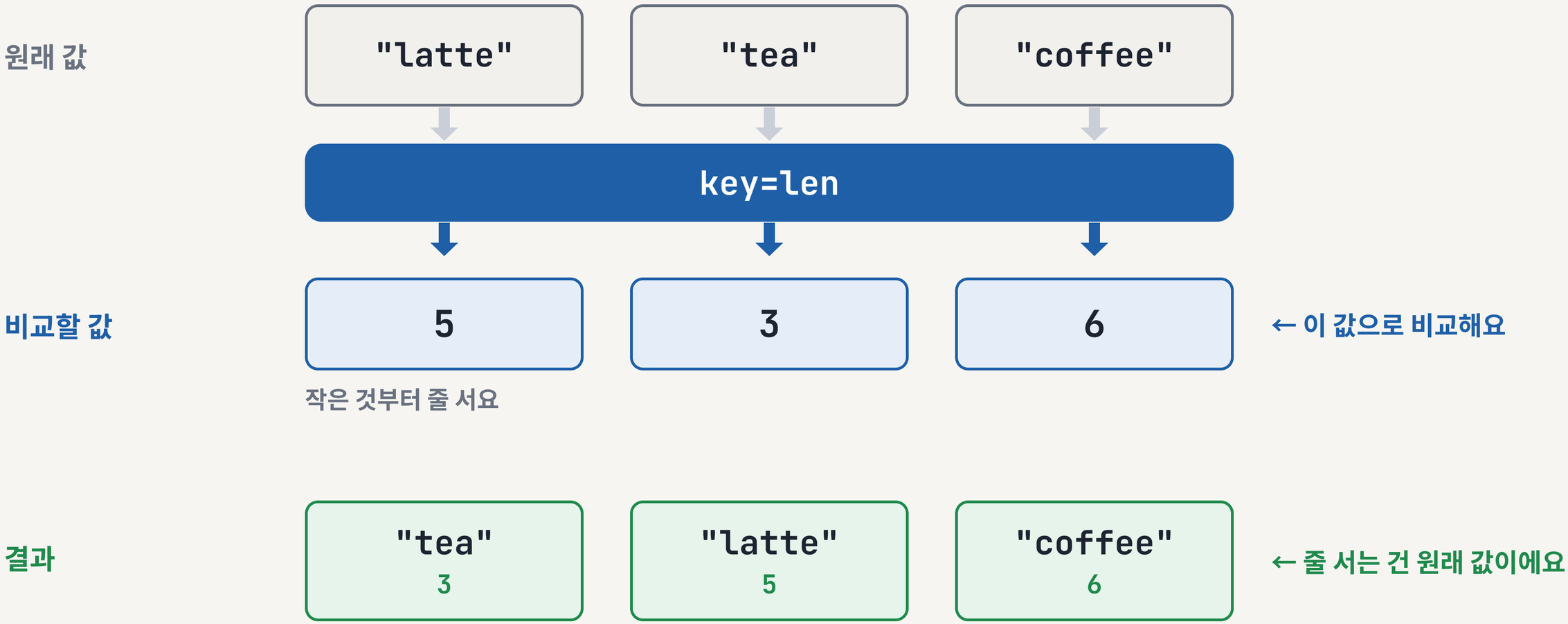
coffee	latte	cake	tea
12	30	8	15

← 판매량은 latte가 30잔으로 가장 많아요

이름 말고 판매량으로 비교하라고 알려 주는 방법이 key예요.

1. 왜 필요한가

key는 비교할 값을 따로 만들어 줘요



가장 짧은 key: 이름을 길이순으로

[2]
```python
names = ["latte", "tea", "coffee"]
print(sorted(names))
print(sorted(names, key=len))
print(max(sales, key=sales.get))
```

```
['coffee', 'latte', 'tea']
['tea', 'latte', 'coffee']
latte
```

← 길이로 비교
← 판매량으로 비교
← 길이 3, 5, 6 순
← 앞 장의 tea가 latte로

21	`map(len, names)`	len을 괄호 없이 넘겼어요
22	`sorted(names, key=len)`	key에도 괄호 없이 넘겨요

2. sorted, sort, key

sorted는 새 리스트, sort는 원본을 바꿔요

[1]
```python
prices = [6500, 4500, 5000]
a = sorted(prices)
print(a)
print(prices)
prices.sort()
print(prices)
```

```
[4500, 5000, 6500]
[6500, 4500, 5000]
[4500, 5000, 6500]
```



원본을 지키려면 sorted, 원본을 바꿔도 되면 sort를 써요.

2. sorted, sort, key

prices = prices.sort()를 하면 None만 남아요

```
[2]  prices = [6500, 4500, 5000]
     prices = prices.sort()
     print(prices)
                                                    Python
     None
```

← sort가 돌려준 값을 prices에 넣었어요

← 줄 세운 리스트를 잃었어요

1	`prices.sort()`	prices를 줄 세워요
2	돌려준 값	None
3	`prices = None`	prices에 None이 들어가요

× ~~`prices = prices.sort()`~~ → ✓ `prices.sort()` 또는 `prices = sorted(prices)`

2. sorted, sort, key

큰 것부터 세우려면 reverse=True

[3]
```python
prices = [6500, 4500, 5000]
print(sorted(prices, reverse=True))
prices.sort(reverse=True)
print(prices)
```

← 20의 키워드 인자

← sort도 똑같이 받아요

```
[6500, 5000, 4500]
[6500, 5000, 4500]
```

기본 | 4500 | 5000 | 6500 |

reverse=True | 6500 | 5000 | 4500 | ← 큰 것부터, 글자라면 z부터

2. sorted, sort, key

글자는 앞 글자부터 하나씩 비교해요

[4]
```python
print(sorted(["Latte", "cake", "Tea", "coffee"]))
print(sorted(["라떼", "커피", "녹차"]))
print(sorted(["10", "9", "100"]))
```

```
['Latte', 'Tea', 'cake', 'coffee']
['녹차', '라떼', '커피']
['10', '100', '9']
```

← 크기순이 아니에요

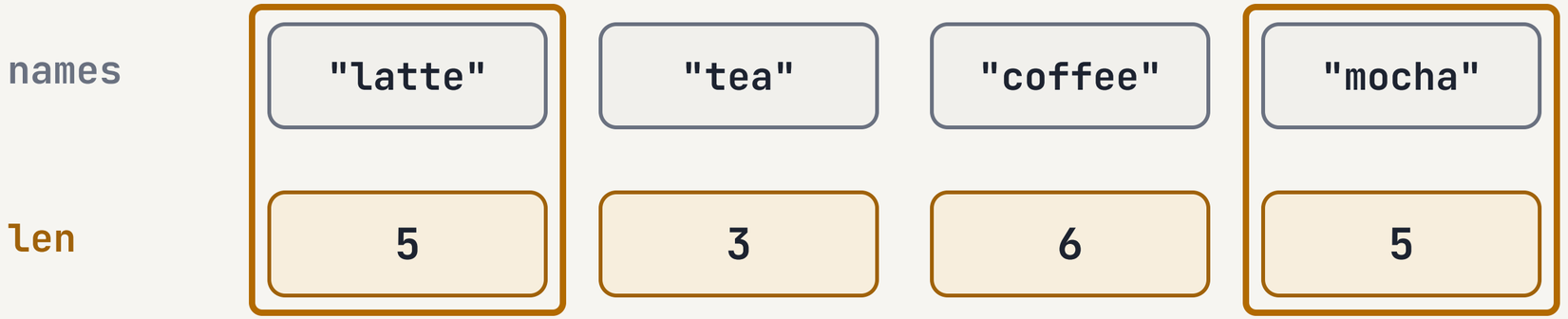
`"Tea" < "cake"`	`True`	대문자 A~Z 뒤에 소문자 a~z가 와요
`"녹차" < "라떼"`	`True`	한글은 가나다 순이에요
`"100" < "9"`	`True`	첫 글자 1과 9를 먼저 비교해요

key=len: 길이를 재서 비교해요

[5]
```python
names = ["latte", "tea", "coffee", "mocha"]
print(list(map(len, names)))
print(sorted(names, key=len))
print(sorted(names, key=len, reverse=True))
```

```
[5, 3, 6, 5]
['tea', 'latte', 'mocha', 'coffee']
['coffee', 'latte', 'mocha', 'tea']
```

← 21의 map(len, ...)과 같은 값



← 길이가 같으면 처음 적은 순서

2. sorted, sort, key

key=str.lower, key=int로 비교 방법을 바꿔요

[6]
```python
print("Latte".lower())
print(sorted(["Latte", "cake", "Tea", "coffee"], key=str.lower))
print(sorted(["10", "9", "100"], key=int))
```

Python

```
latte
['cake', 'coffee', 'Latte', 'Tea']
['9', '10', '100']
```

`"Latte".lower()`	글자를 모두 소문자로 바꾼 새 글자를 줘요
`key=str.lower`	소문자로 바꿔서 비교해요. 결과는 원래 글자예요
`key=int`	숫자로 바꿔서 크기순으로 비교해요

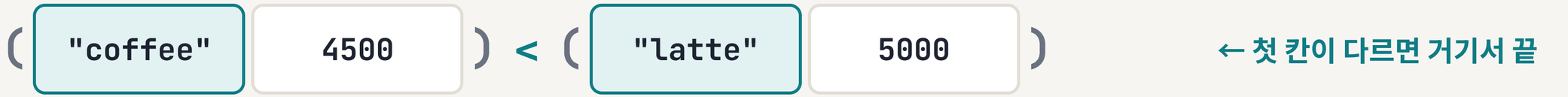
3. 튜플과 딕셔너리 정렬

튜플은 앞 칸부터 비교해요

[1]
```python
items = [("latte", 5000), ("cake", 6500), ("coffee", 4500), ("cake", 6000)]
print(sorted(items))
```

```
[('cake', 6000), ('cake', 6500), ('coffee', 4500), ('latte', 5000)]
```





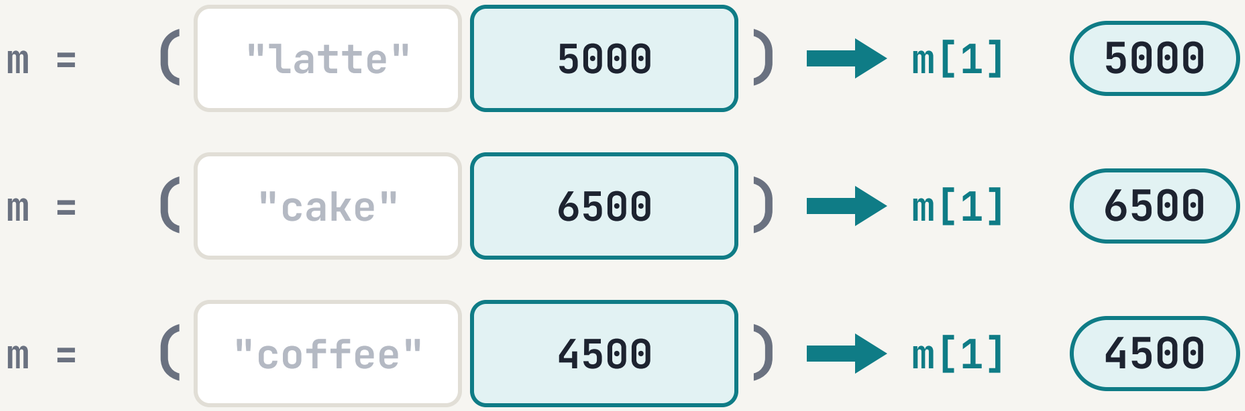
글자를 첫 글자부터 비교하듯, 튜플은 첫 칸부터 비교해요.

key=lambda m: m[1]로 둘째 칸 기준

[2]
```python
items = [("latte", 5000), ("cake", 6500), ("coffee", 4500)]
print(sorted(items, key=lambda m: m[1]))
print(sorted(items, key=lambda m: m[1], reverse=True))
```

← m에 튜플이 하나씩

```
[('coffee', 4500), ('latte', 5000), ('cake', 6500)]
[('cake', 6500), ('latte', 5000), ('coffee', 4500)]
```



← 가격으로 비교하고, 결과는 튜플 그대로

3. 튜플과 딕셔너리 정렬

판매량 많은 순, 같으면 이름순으로 세워요

[3]
```python
sales = [("latte", 30), ("cake", 8), ("coffee", 30), ("tea", 15)]
print(sorted(sales, key=lambda m: m[1], reverse=True))
print(list(map(lambda m: (-m[1], m[0]), sales)))
print(sorted(sales, key=lambda m: (-m[1], m[0])))
```

```
[('latte', 30), ('coffee', 30), ('tea', 15), ('cake', 8)]
[(-30, 'latte'), (-8, 'cake'), (-30, 'coffee'), (-15, 'tea')]
[('coffee', 30), ('latte', 30), ('tea', 15), ('cake', 8)]
```

← 30끼리는 원래 순서

← 같으면 이름순

튜플	key 결과	읽는 법
("coffee", 30)	(-30, 'coffee')	-30이 가장 작고, 이름도 c가 먼저
("latte", 30)	(-30, 'latte')	-30이 같아서 이름으로 비교해요
("tea", 15)	(-15, 'tea')	다음은 -15
("cake", 8)	(-8, 'cake')	-8이 가장 커서 맨 뒤

딕셔너리는 키, 값, 짝 중 무엇을 줄 세울지 골라요

[4]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales))
print(sorted(sales.values()))
print(sorted(sales.items()))
```

```
['cake', 'coffee', 'latte', 'tea']
[8, 12, 15, 30]
[('cake', 8), ('coffee', 12), ('latte', 30), ('tea', 15)]
```

sorted(sales)

coffee	latte	cake	tea
12	30	8	15

키만, 이름이 abc 순으로

sorted(sales.values())

coffee	latte	cake	tea
12	30	8	15

값만, 메뉴 이름은 빠져요

sorted(sales.items())

coffee	latte	cake	tea
12	30	8	15

(키, 값) 튜플, 첫 칸 이름순

items()에 key=lambda kv: kv[1]로 판매 순위

[5]
```python
rank = sorted(sales.items(), key=lambda kv: kv[1], reverse=True)
print(rank)
for name, n in rank:
    print(f"{name:<7}{n:>3}잔")
```

← 판매량으로 비교

```
[('latte', 30), ('tea', 15), ('coffee', 12), ('cake', 8)]
latte   30잔
tea     15잔
coffee  12잔
cake     8잔
```

← 13의 꺼내 담기



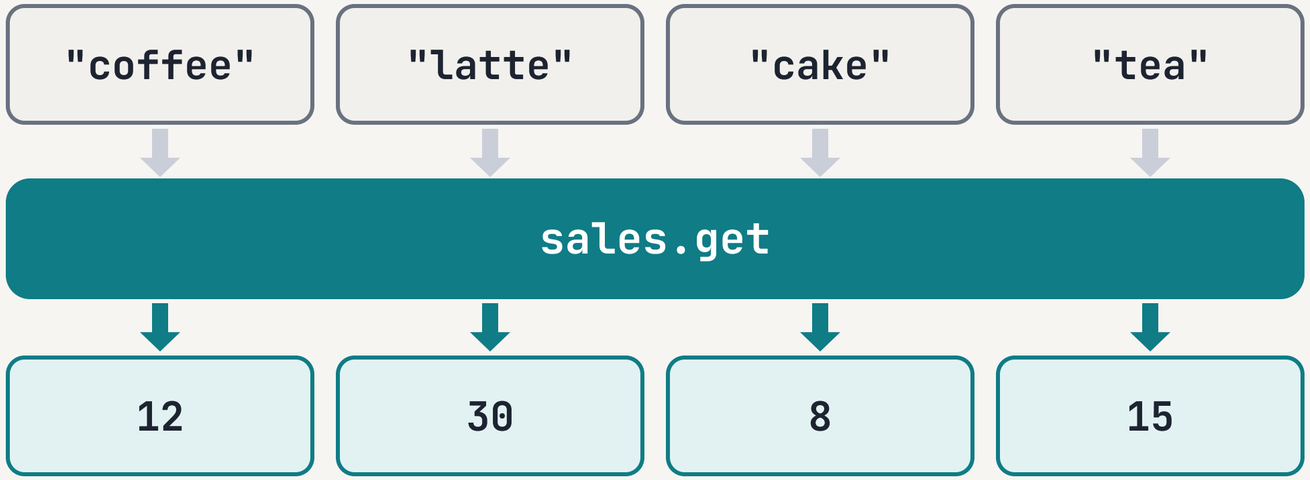
이름과 값을 같이 두고 값으로 세우려면 items()에 key를 줘요.

키만 필요하면 key=sales.get

[6]
```python
print(sales.get("latte"))
print(sorted(sales, key=sales.get))
print(sorted(sales, key=sales.get, reverse=True))
```

```
30
['cake', 'coffee', 'tea', 'latte']
['latte', 'tea', 'coffee', 'cake']
```

← 15의 get, 키를 넣으면 값



← 괄호 없이 이름만 넘겨요

← lambda k: sales[k]와 같아요

4. max와 min

max와 min도 key를 받아요

[1]
```python
prices = [4500, 5000, 6500, 3000]
print(max(prices), min(prices))
names = ["latte", "tea", "coffee"]
print(max(names), min(names))
print(max(names, key=len))
```

```
6500 3000
tea coffee
coffee
```

← 글자는 abc 순, sorted와 같은 규칙

← 길이 6이 아니라 이름 coffee

sorted(names)



← 맨 앞이 min, 맨 뒤가 max

4. max와 min

가장 많이 팔린 메뉴는 max(sales, key=sales.get)

[2]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(max(sales))
print(max(sales, key=sales.get))
print(min(sales, key=sales.get))
print(max(sales.items(), key=lambda kv: kv[1]))
```

```
tea
latte
cake
('latte', 30)
```

← 1파트의 tea

← 판매량이 가장 많은 키

← 판매량이 가장 적은 키

← 이름과 판매량을 같이

coffee	latte	cake	tea
12	30	8	15

4. max와 min

빈 리스트에 max를 쓰면 ValueError가 나요

[3]
```python
print(max([]))
```

```
ValueError: max() iterable argument is empty
```

← 고를 값이 하나도 없어요

[4]
```python
big = list(filter(lambda p: p > 10000, prices))
print(big)
print(max(big, default=0))
```

```
[]
0
```

← 1만 원 넘는 메뉴가 없어요

← 비어 있으면 0

15	`menu.get("cakee", 5000)`	키가 없을 때 대신 돌려줄 값
22	`max(big, default=0)`	비어 있을 때 대신 돌려줄 값

4. max와 min

sum, len, 자르기와 같이 써요

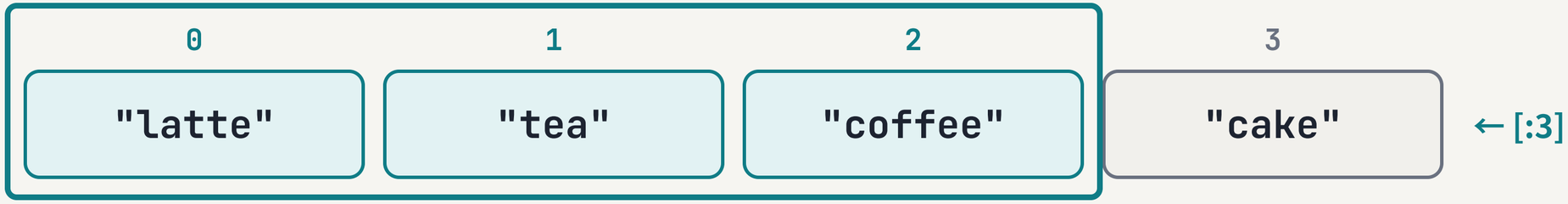
[5]
```python
print(sum(sales.values()), len(sales))
print(sum(sales.values()) / len(sales))
top3 = sorted(sales, key=sales.get, reverse=True)[:3]
print(top3)
```

```
65 4
16.25
['latte', 'tea', 'coffee']
```

← 모두 65잔, 메뉴 4개

← 메뉴 하나당 평균 16.25잔

← 줄 세운 리스트의 앞 3개



← [:3]

5. 오류와 함정

숫자와 글자를 섞으면 비교할 수 없어요

[1]
```python
print(sorted([4500, "5000", 6500]))
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```
← 글자와 숫자

[2]
```python
print(sorted([4500, "5000", 6500], key=int))
```

```
[4500, '5000', 6500]
```
← 비교할 때만 숫자로

← "5000"은 그대로 글자

× ~~[4500, "5000", 6500]~~ → ✓ `[4500, 5000, 6500]`

한 리스트에는 같은 타입을 담아요. 섞였으면 key로 맞춰 비교해요.

key에 괄호를 붙이면 TypeError가 나요

[3]
```python
print(sorted(["latte", "tea"], key=len()))
```

```
TypeError: len() takes exactly one argument (0 given)
```

[4]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales, key=sales.get()))
```

```
TypeError: get expected at least 1 argument, got 0  ← 아무것도 안 넣고 먼저 불렸어요
```

✕ ~~key=len()~~ → ✓ `key=len`

✕ ~~key=sales.get()~~ → ✓ `key=sales.get`

lambda나 key=를 빠뜨리면 생기는 오류

[5]
```python
print(sorted(sales.items(), key=kv[1]))
```
← lambda kv: 가 빠졌어요

```
NameError: name 'kv' is not defined
```

[6]
```python
print(sorted(sales, sales.get))
```
← key= 이름이 빠졌어요

```
TypeError: sorted expected 1 argument, got 2
```

× ~~`key=kv[1]`~~ → ✓ `key=lambda kv: kv[1]`

× ~~`sorted(sales, sales.get)`~~ → ✓ `sorted(sales, key=sales.get)`

key는 꼭 key=로, lambda는 lambda 이름: 까지 적어요.

6. 실습

판매 순위표 3위까지 찍기

[1]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15, "juice": 15}
rank = sorted(sales.items(), key=lambda kv: (-kv[1], kv[0]))
for i, kv in enumerate(rank[:3], 1):
    print(f"{i}위 {kv[0]:<6}{kv[1]:>3}잔")
```

← 두 기준 (3파트)
← 1위부터 (18)

```
1위 latte  30잔
2위 juice  15잔
3위 tea    15잔
```

key 결과 (-15 'juice') < (-15 'tea') ← -15가 같아서 이름으로, juice가 먼저

줄 세우기, 자르기, 번호 붙이기를 이어서 순위표를 만들어요.

6. 실습

가장 비싼 메뉴, 가장 싼 메뉴, 가격순 메뉴판

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
hi = max(menu, key=menu.get)
lo = min(menu, key=menu.get)
print(f"가장 비싼 메뉴 {hi} {menu[hi]:,}원")
print(f"가장 싼 메뉴 {lo} {menu[lo]:,}원")
for m in sorted(menu, key=menu.get):
    print(f"{m:<7}{menu[m]:>6,}원")
```

Python

← 이름을 받아요

← menu[hi]로 가격 꺼내기

← 가격순 키 리스트

```
가장 비싼 메뉴 cake 6,500원
가장 싼 메뉴 tea 4,000원
tea     4,000원
coffee  4,500원
latte   5,000원
cake    6,500원
```

6. 실습

메뉴판을 이름 길이순, 같으면 abc 순으로

[3]
```python
names = ["mocha", "tea", "latte", "americano", "cake"]
print(sorted(names, key=len))
print(sorted(names, key=lambda n: (len(n), n)))
```

```
['tea', 'cake', 'mocha', 'latte', 'americano']
['tea', 'cake', 'latte', 'mocha', 'americano']
```

← 5끼리는 원래 순서

← 같으면 abc 순

key 결과 (5 , "latte") < (5 , "mocha") ← 첫 칸 5가 같아서 이름으로

판매 순위 (3파트)	`(-m[1], m[0])`	많은 순, 같으면 이름순
메뉴판 (지금)	`(len(n), n)`	짧은 순, 같으면 abc 순

6. 실습

성적표를 점수순으로 세우고 1등과 평균 구하기

[4]
```python
scores = [("kim", 92), ("lee", 78), ("park", 85), ("choi", 59)]
ranked = sorted(scores, key=lambda s: s[1], reverse=True)
for name, score in ranked:
    print(f"{name:<5}{score:>3}")
print("1등:", max(scores, key=lambda s: s[1])[0])
print("평균:", sum(map(lambda s: s[1], scores)) / len(scores))
```

← 점수 높은 순

← 튜플에서 이름만

← 21의 map으로 점수만

```
kim   92
park  85
lee   78
choi  59
1등: kim
평균: 78.5
```

정리

막히면 오류의 마지막 줄을 봐요

1	max(sales)가 엉뚱한 이름이에요	딕셔너리는 키끼리 비교해요	`key=sales.get`
2	prices를 찍으니 None이에요	prices = prices.sort()로 받았어요	**prices.sort()만**
3	['10', '100', '9'] 순서예요	글자는 앞 글자부터 비교해요	`key=int`
4	`TypeError: '<' not supported ...`	숫자와 글자를 섞었어요	**타입 맞추기, key=int**
5	`TypeError: len() takes exactly ...`	key에 괄호를 붙였어요	`key=len`
6	`NameError: name 'kv' is not defined`	lambda kv: 를 빠뜨렸어요	`key=lambda kv: kv[1]`
7	`ValueError: max() iterable ... empty`	빈 리스트에 max를 썼어요	`default=0`

순서가 이상하면 map(key 함수, 리스트)로 비교할 값을 먼저 찍어 봐요.

한 장 요약

`sorted(xs)`
새 리스트, 원본 그대로

`xs.sort()`
원본을 바꾸고 None

`key=len`
len 결과로 비교

`key=lambda m: m[1]`
튜플 둘째 칸으로

`key=lambda m: (-m[1], m[0])`
많은 순, 같으면 이름순

`sorted(d, key=d.get)`
값 순서로 키 세우기

`max(d, key=d.get)`
값이 가장 큰 키

`max(xs, default=0)`
비어 있으면 0

key에는 괄호 없이 함수 이름이나 lambda를 넣어요.

다음 챕터 23에서는 리스트 컴프리헨션으로 고르고 바꾸기를 한 줄에 적어요.